

# **Modular Sort Interface for 4D (ORDA and Classic Mode)**

By Olivier Marolleau, Quality Support Engineer, 4D France

Technical Note 26-07

## Table of Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Table of Contents .....                                     | 2  |
| Abstract.....                                               | 4  |
| Introduction .....                                          | 4  |
| Prerequisites .....                                         | 4  |
| System Overview.....                                        | 4  |
| Application Purpose .....                                   | 4  |
| Context.....                                                | 4  |
| Objective .....                                             | 5  |
| Value Added .....                                           | 5  |
| Use Cases .....                                             | 5  |
| Application Workflow .....                                  | 5  |
| Key Features .....                                          | 6  |
| Opening the Sort Interface.....                             | 6  |
| Usage in Classic Mode .....                                 | 6  |
| Usage with ORDA .....                                       | 6  |
| Module System and Sort Composition .....                    | 7  |
| Searching and Selecting Modules .....                       | 9  |
| Dialog Actions .....                                        | 11 |
| Technical Architecture.....                                 | 11 |
| Overview .....                                              | 12 |
| Central Class .....                                         | 12 |
| Main Form .....                                             | 12 |
| Embedded Resources .....                                    | 13 |
| Solution Autonomy .....                                     | 13 |
| Technologies Used.....                                      | 13 |
| 4D Language .....                                           | 13 |
| Embedded Web Interface .....                                | 13 |
| 4D ⇌ Web Communication .....                                | 13 |
| JavaScript Libraries .....                                  | 13 |
| Key Technical Points / Architecture Choices .....           | 14 |
| Extending the TRI Command for Modern Sorting Scenarios..... | 14 |
| Modular Representation of Sorting .....                     | 14 |
| Separation Between Sort Definition and Execution .....      | 15 |
| General System Operation.....                               | 15 |
| Interaction with the Current Selection.....                 | 16 |
| Internationalization of the Web Interface .....             | 17 |
| Current Limitations .....                                   | 17 |
| Future Feature Enhancements .....                           | 17 |
| Description of the Hub (Lib_Mod2).....                      | 18 |
| localizedString .....                                       | 18 |
| isClassicMode .....                                         | 18 |
| orderBy .....                                               | 18 |
| paletteLoadDictionary .....                                 | 18 |
| buttonCloseWin.....                                         | 19 |
| buttonPlansLoad .....                                       | 19 |
| buttonPlansSave .....                                       | 19 |
|                                                             | 2  |

|                         |    |
|-------------------------|----|
| buttonPlansExecute..... | 19 |
| Conclusion .....        | 19 |

## Abstract

This technical note presents a modular sort interface developed for 4D, enabling dynamic, configurable, and reusable sorting for ORDA's object mode. The solution allows developers to create and run multi-criteria sorts through a graphical interface, with no coding required.

Built on a modular architecture, the interface works equally well with classic selections and ORDA entity selections. It maintains a clean separation between the user interface and business logic. Sort configurations can also be saved and reloaded, simplifying the creation of complex sorts and improving their reusability and maintainability.

## Introduction

The evolution of 4D with the arrival of ORDA has profoundly changed the way data is manipulated. While this approach brings great flexibility, it also introduces certain differences compared with the language's historical mechanisms. In particular, ORDA does not natively include a sort editor that lets users build complex, multi-criteria sorts dynamically on an entity selection — a capability developers previously took for granted with classic selections.

This technical note presents a solution designed to fill this gap by means of a fully modular graphical interface. Users can build one or more sort criteria visually, without writing any 4D code, and then apply these criteria to either a classic selection or an ORDA entity selection.

Beyond the user interface itself, this document describes the architectural choices made, the technologies used, and the mechanisms implemented to ensure communication between the business logic developed in 4D and the HTML/JavaScript interface. The goal is to offer a generic, extensible solution that can be easily integrated into an existing 4D application.

## Prerequisites

This library is compatible with 4D version 21 R3 or any later version.

The project relies on the features introduced with 4D classes, ORDA, and modern Web Areas. General knowledge of these technologies is recommended to understand the internal mechanisms described in this documentation.

## System Overview

### Application Purpose

#### **Context**

4D's `ORDER BY` command has long been a simple way to sort a classic selection. However, ORDA entity selections do not provide the same mechanism for dynamically building sort criteria.

This becomes particularly challenging when an application needs to offer users complex, configurable, or reusable sorting capabilities.

## Objective

The project's goal is to provide a graphical sort editor capable of working independently of the data manipulation mode.

Users build their sort using visual modules that represent the different elements making up a criterion (table, field, sort direction, etc.). Once the configuration is complete, it is interpreted and automatically applied to the current context, whether that is a classic selection or an ORDA entity selection.

## Value Added

This approach offers several advantages:

- Creating multi-criteria sorts without programming.
- Support for both classic and ORDA selections from a single interface.
- Saving and reloading sort configurations.
- A modular architecture that facilitates future development.
- A complete separation between the user interface and the business logic.

## Use Cases

The interface addresses the following needs in particular:

- Building a multi-criteria sort on a classic selection;
- Applying the same criteria to an ORDA entity selection;
- Easily changing the priority order of the criteria;
- Saving sort plans so they can be reused later;

Sharing a sort configuration between several users or several applications.

**Note :** *Support for sorts based on formulas or custom expressions is not yet available. This feature is part of the planned future developments.*

## Application Workflow

The application's operation is based on a principle of modular composition. Rather than manually entering sort criteria or writing code, the user builds their sort graphically by assembling different modules within the interface.

Each module represents a building block of a sort criterion, such as a table, a field, or a sort direction (ascending or descending). By combining them one after another, the user creates one or more criteria that precisely describe the desired sort order.

Once the composition is complete, the full set of modules is converted into an internal data structure representing the complete sort definition. This structure is then interpreted by the application's engine to automatically apply the defined criteria to the current context, whether that is a classic selection or an ORDA entity selection.

The interface continuously applies validation rules that guarantee the consistency of the criteria being built. Each module has positioning constraints that determine which elements can precede or follow it. This makes it impossible to assemble incompatible modules or create an invalid sort structure.

For example, a module representing a field cannot be placed immediately after another field-type module unless a table or other context information has already been defined beforehand. Similarly, a sort direction can only be added after a valid field.

This real-time validation naturally guides the user in building their criteria and considerably reduces the risk of error, while also making the interface intuitive, even for users who are not familiar with the 4D language.

## Key Features

### Opening the Sort Interface

Sort criteria are built within a dedicated dialog that offers a graphical interface based on a system of drag-and-drop modules. Opening this interface is identical regardless of the context of use. Only the nature of the parameter passed to the **orderBy()** method differs depending on whether the application is working with a classic selection or an ORDA entity selection.

#### Usage in Classic Mode

In the case of a classic selection, the method receives a pointer to the relevant table:

```
cs.Lib_Mod2.me.orderBy(Current form's table)
```

The pointer allows the library to identify the table on which the sort will be applied. At execution time, the TRI command acts directly on the current selection of that table.

#### Usage with ORDA

In an ORDA context, the method instead receives a reference to an entity selection, along with the path used to locate that reference within the form:

```
cs.Lib_Mod2.me.orderBy(Form.dataExplore.exSelection; "Form.dataExplore.exSelection")
```

This combination of information allows the sort engine to directly manipulate the original entity selection and update its contents once the sort has been executed.

Despite this difference in internal behavior, the user sees the same graphical interface. The data access mode remains completely transparent, guaranteeing an identical experience in both classic and ORDA mode.

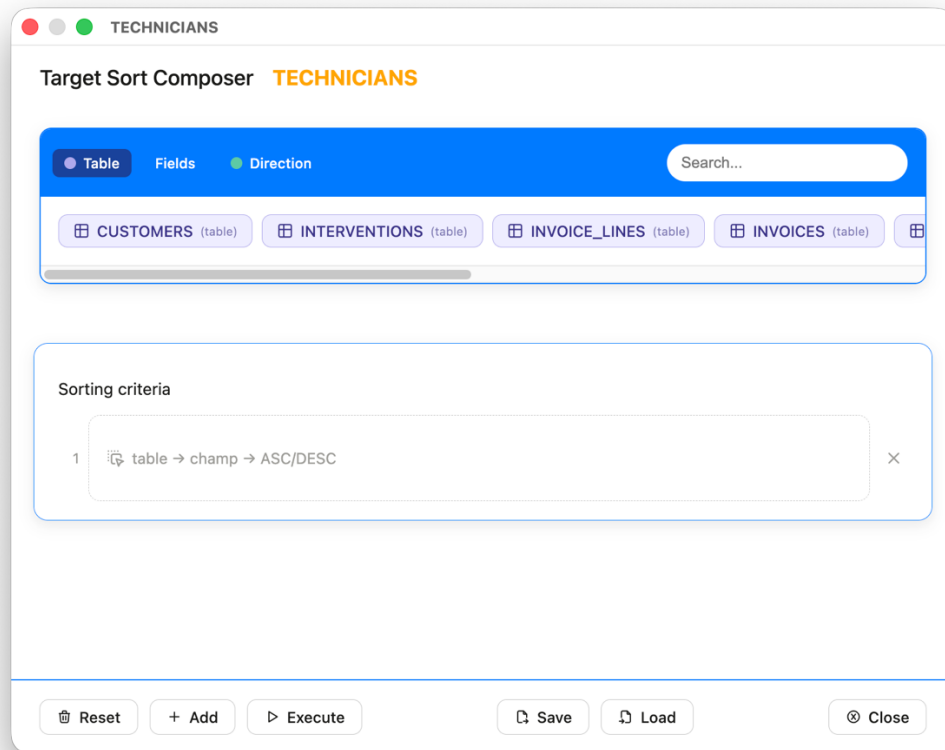


Figure 1: Main interface of the sort editor

### Module System and Sort Composition

The editor is based on a system of modules, each one representing an element of a sort definition.

To build a criterion, the user selects the desired modules from the palette and then drops them into the composition area with a simple drag-and-drop action.

Creating a criterion generally follows these steps:

- Selecting the relevant table.
- Selecting the field to use.
- Selecting the sort direction (ASC or DESC).

For example, a criterion can consist of the following elements: the table “TECHNICIANS”, then the field “Name”, followed by the direction “ASC”.

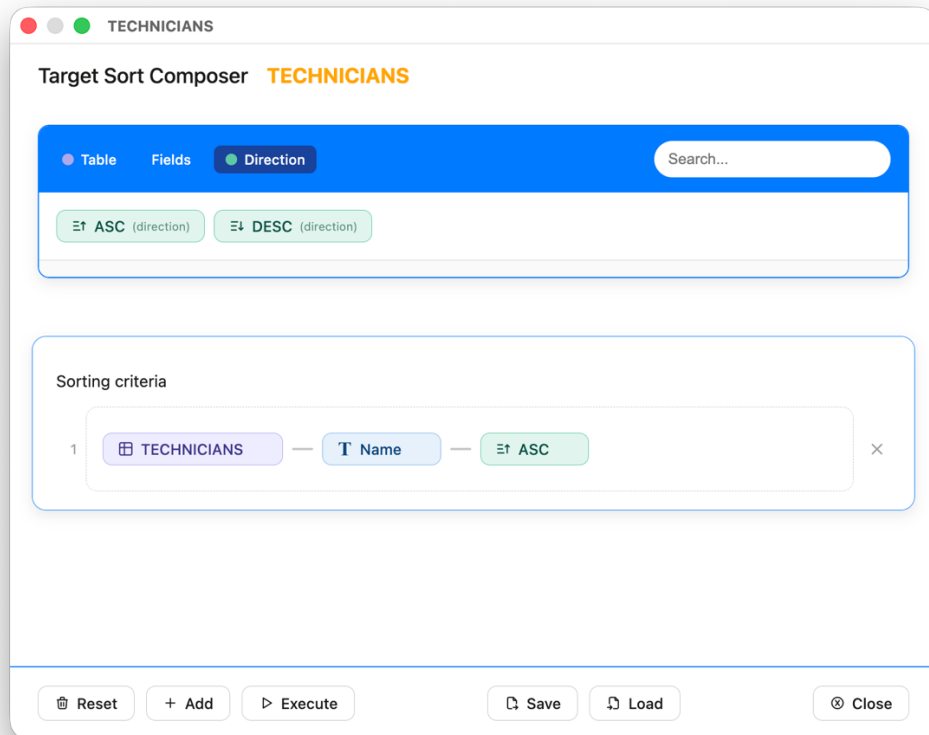
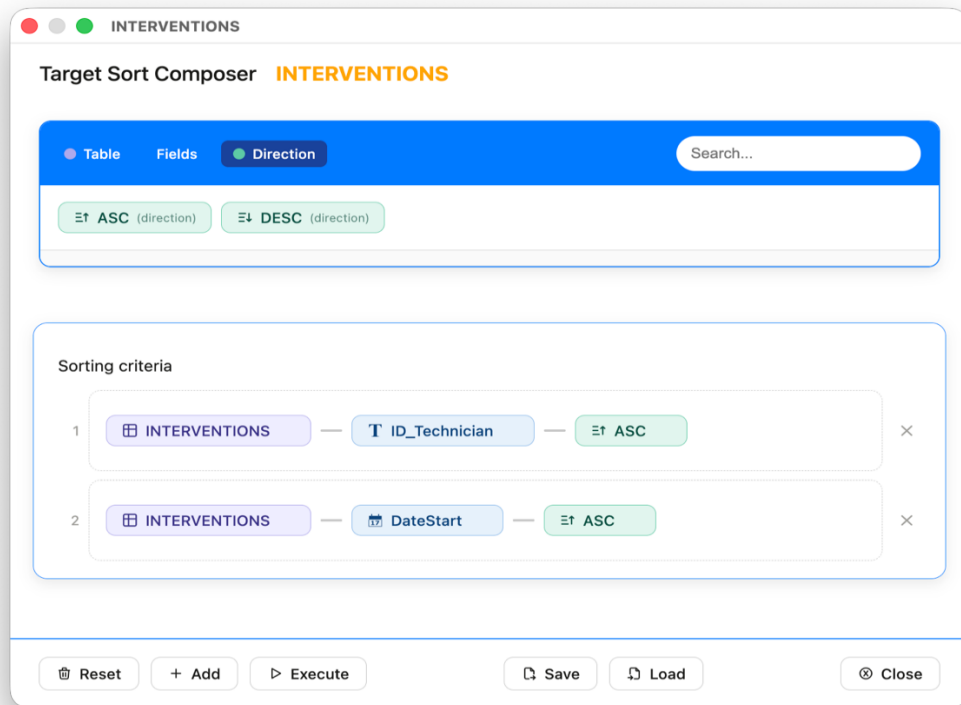


Figure 2: example of a sort on a single criterion

Each module added progressively completes the definition of the criterion. Several criteria can then be created to build a multi-criteria sort, with each criterion representing an additional level of priority.

The interface automatically validates the resulting structure to prevent the creation of inconsistent criteria.



*Figure 2b: example of a multi-criteria sort*

## Searching and Selecting Modules

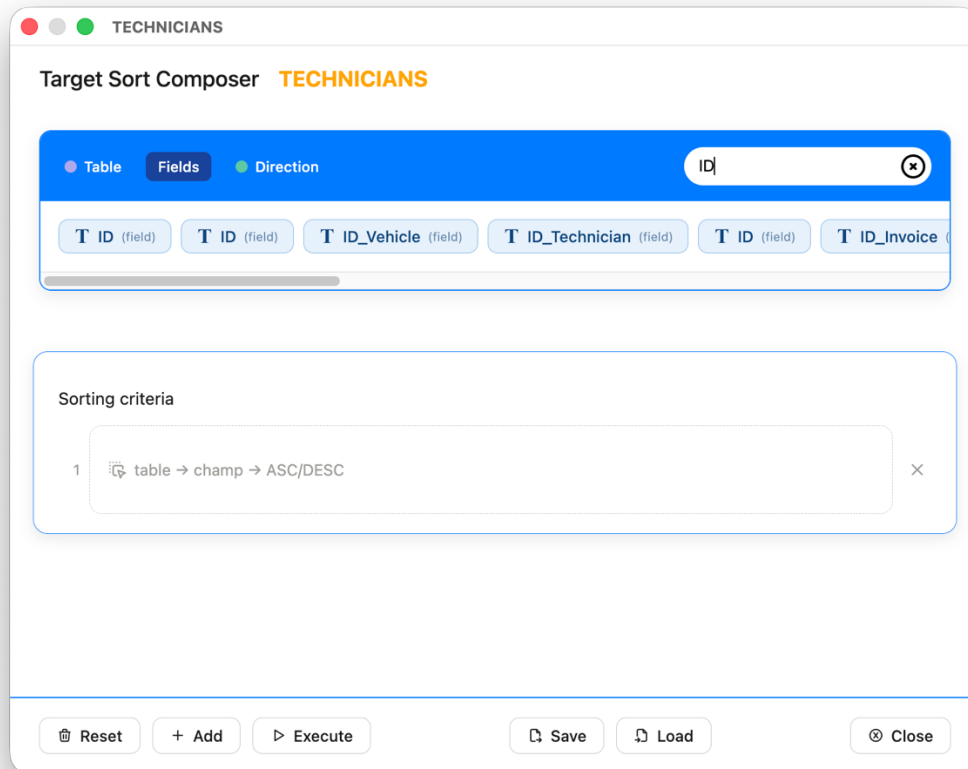
When the number of tables or fields becomes large, finding a particular element can quickly become tedious.

To simplify this step, each category of modules has its own search field. This field dynamically filters the displayed elements as the user types.

Currently, three categories of modules are available:

- Table
- Field
- Direction

Filtering is applied only to the currently selected category, which makes it possible to quickly find an element even in a database containing many tables or fields.

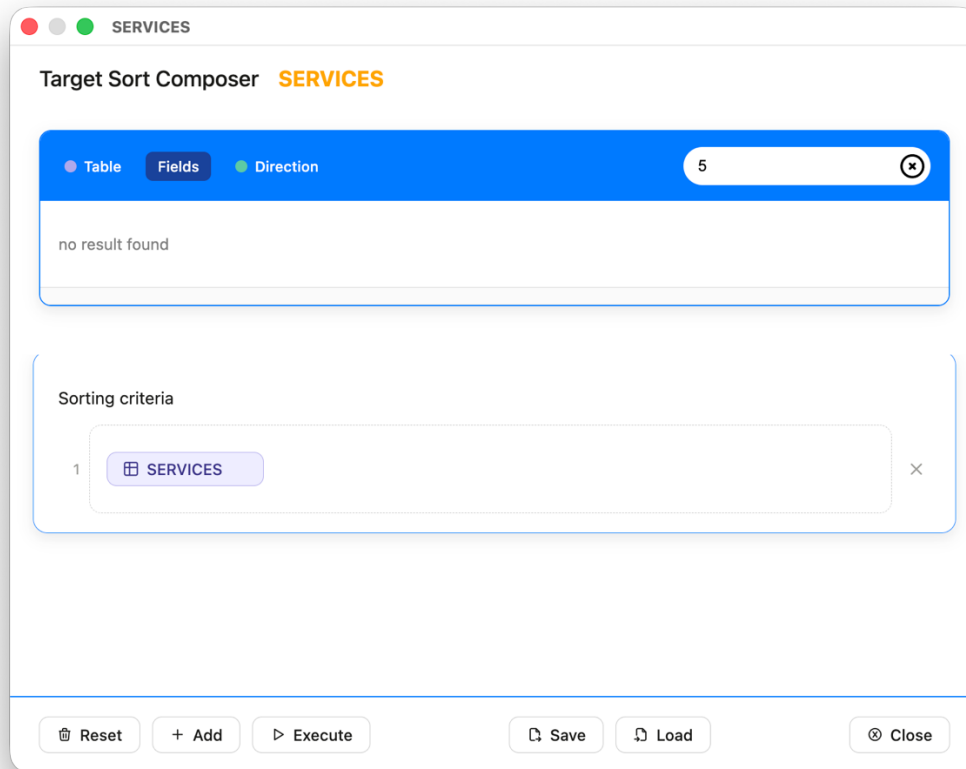


*Figure 3: Filtering fields containing ID*

Hovering the mouse over a module also displays additional information as a tooltip, helping the user quickly identify the module they are looking for.

The interface also offers contextual navigation. For example, once a table has been added to a criterion, simply clicking on it automatically selects the **Field** category and filters the list to show only the fields belonging to that table.

This assistance considerably reduces the number of steps required and speeds up the creation of sort criteria.



*Figure 4: Automatic filtering of fields for the selected table*

## Dialog Actions

Several commands are available at the bottom of the window to manage the sort criteria.

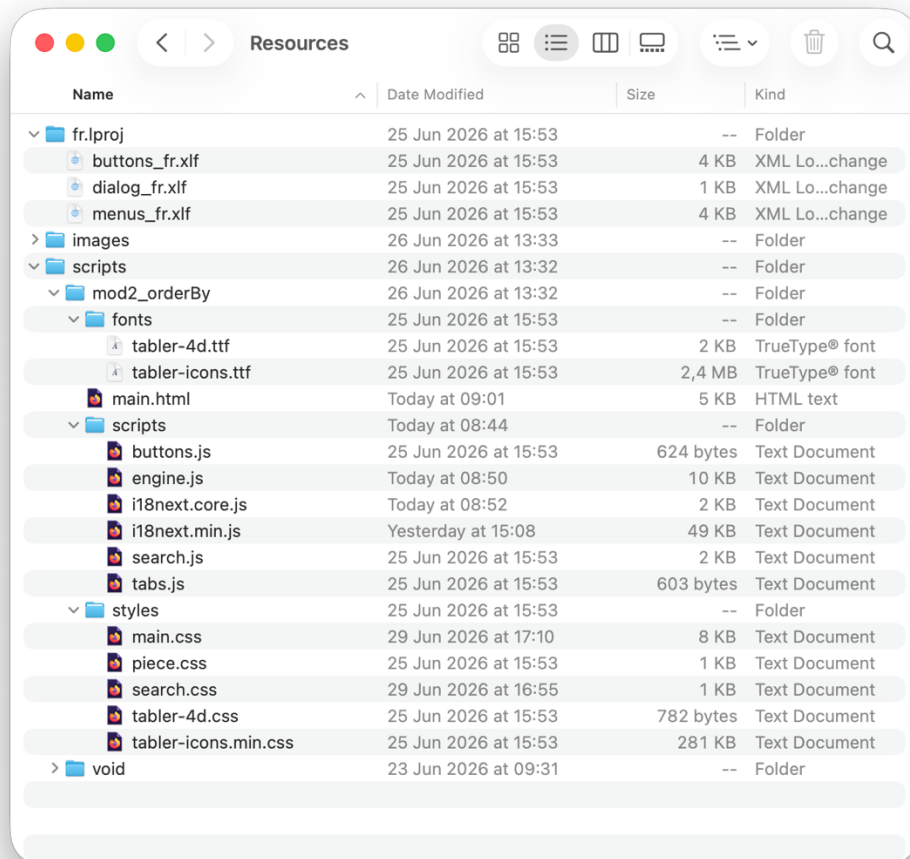
- **Reset:** deletes all currently defined criteria and resets the editor.
- **Add:** creates a new sort criterion. This feature makes it possible to build multi-criteria sorts by defining several priority levels.
- **Execute:** immediately applies the sort criteria to the current selection. Unlike 4D's native sort editor, the dialog remains open after execution, making it possible to test different configurations without having to reopen the window.
- **Save:** saves the complete sort definition to a file in **.4od** format for later reuse.

**Load:** reloads a previously saved **.4od** file and automatically restores the full set of sort criteria in the editor.

## Overview

The application is built on a hybrid architecture combining 4D's native capabilities with an embedded web interface. This organization makes it possible to clearly separate the business logic, the user interface, and the rendering resources.

The project is structured around several main components:



### Central Class

The `Lib_Mod2` class is the core of the application. It brings together all the business logic needed to build, manage, and execute sort criteria. It acts as a genuine functional entry point (hub) between the user interface and the 4D engine.

### Main Form

The project form `fm_mod2_orderBy` serves as the main container for the interface. It hosts a Web Area into which the HTML/JavaScript application responsible for rendering the sort editor is loaded.

This form also acts as an interface between the 4D world and the presentation layer.

## Embedded Resources

Two folders located in the resources folder complete the architecture:

- **scripts/**: contains the entire JavaScript engine and the scripts required for the sort interface to function. This folder groups together the rendering logic, module management, and user interactions.
- **x.lproj/**: contains the localization files used to translate the interface. This structure ensures multilingual support for the application.

## Solution Autonomy

All the libraries required for the application to run are embedded locally within the project. No external component is required at runtime, guaranteeing autonomous, stable execution that is independent of any third-party connection or dependency.

## Technologies Used

The solution relies on a combination of complementary technologies that make the most of both the robustness of 4D and the flexibility of the web.

### 4D Language

The application core is developed in modern 4D language, structured around the **Lib\_Mod2** class. This class centralizes business-logic calls and coordinates the different components of the system.

### Embedded Web Interface

The user interface is built entirely in **HTML** and **JavaScript**. It is displayed in a Web Area embedded in the 4D form.

This approach provides a rich, dynamic, and easily maintainable interface, while preserving native integration with the 4D environment.

### 4D ⇄ Web Communication

Data exchange between 4D and the Web Area relies on the **\$4d** mechanism, which allows JavaScript to invoke 4D methods directly.

This mechanism is the bridge between the presentation layer and the business logic. The data exchanged is serialized, generally in text or JSON format, and then reconstructed on the target side.

### JavaScript Libraries

To improve the user experience and simplify the development of the interface, several external libraries are integrated:

- **i18n (internationalization)**: used to manage the translation of the interface. It allows labels to be dynamically adapted based on the browser language or the user's configuration.
- **Tabler Icons**: a vector icon library used to visually enrich the interface without relying on numerous static images.
- **Tabler Icons 4D**: a customized variant of Tabler Icons, adapted to the application's specific needs, notably for the visual representation of field types and business elements.

This combination of technologies ensures an interface that is lightweight, consistent, and easy to extend.

## Key Technical Points / Architecture Choices

### Extending the TRI Command for Modern Sorting Scenarios

4D's native ORDER BY command has historically been a simple and effective mechanism for sorting a selection in classic mode. However, ORDA-based contexts call for a more flexible approach.

The command only supports classic selections and cannot be applied directly to an entity selection. Preventing its use in object-oriented architecture.

In addition, the TRI command operates as an immediate sorting instruction: once executed, the sort criteria are not retained as a reusable configuration.

The approach adopted in this project complements this mechanism by introducing a data-mode-independent representation of sorting, capable of working in both classic and ORDA mode, while also adding criteria persistence.

### Modular Representation of Sorting

One of the project's key architectural decisions is to represent a sort as **independent modules**.

Although the TRI command is easy to use, it becomes unsuitable as complexity grows or when data comes from the ORDA object model. Selecting a field from many entities can quickly become tedious and difficult to manage.

The modular approach makes it possible to break the definition of a sort down into simple elements, each representing a functional unit:

- A table or data context,
- A field,
- A sort direction.

This representation significantly improves the readability, maintainability, and above all the flexibility of the system. It also makes it possible to plan future developments without calling the existing architecture into question.

## Separation Between Sort Definition and Execution

Another key point of architecture is the strict separation between:

- The **sort definition** (user interface).
- Its **execution** (4D business logic).

This separation guarantees complete independence between the UI and the processing engine.

The sort is defined as a modular structure, then serialized into a platform-independent format (a .4od file). This format can then be reused regardless of the execution context, whether that is a classic selection or an ORDA entity selection.

This approach improves the portability of sort configurations and simplifies their maintenance.

## General System Operation

As mentioned earlier, the `Lib_Mod2` class acts as the **central hub** of the application.

The user interface, developed in HTML/JavaScript, contains almost no business logic. It is limited to:

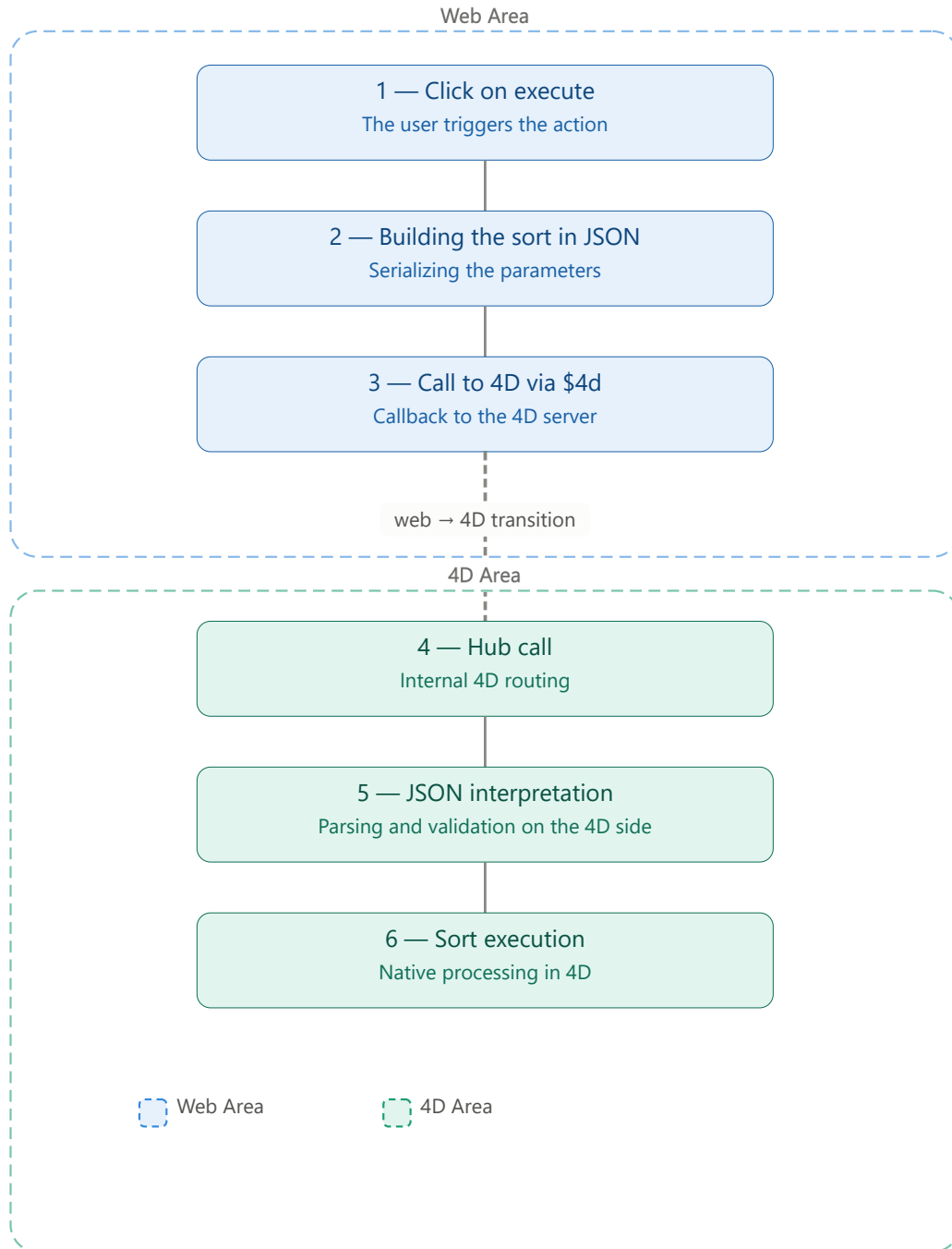
- Querying the hub,
- Displaying the responses,
- Forwarding user actions.

Exchanges between the Web Area and 4D take place via the `$4d` mechanism, which allows 4D methods to be invoked directly from JavaScript.

The overall flow can be summarized as follows:

1. The user interacts with the web interface.
2. A request is sent to 4D via `$4d`.
3. The corresponding `Lib_Mod2` method is executed.
4. 4D processes the request and returns a response (often in JSON format).
5. The interface updates the display accordingly.

*Note: It is important to note that all parameters passed between the two environments are transmitted as text. A serialization/deserialization step is therefore always required in order to rebuild the objects on the 4D side or the JavaScript side.*



*Figure 5: execution example*

### Interaction with the Current Selection

An important point of architecture concerns the application's ability to act on the current selection, whether in a classic or an ORDA context.

When the dialog is opened, the application stores the original context. This information makes it possible to keep a reference to the calling window or process.

Thus, when the user confirms the execution of the sort, the system can reapply the changes directly to the relevant selection.

Updating the context requires using the **EXECUTE FORM** command, which reactivates the original form and synchronizes the display with the sorted data.

### Internationalization of the Web Interface

Localization of interfaces in 4D is traditionally based on the XLIFF mechanism, which is perfectly suited to native forms. However, this mechanism cannot be used directly within an embedded Web Area.

Without a suitable solution, the interface could end up being displayed in a different language from the rest of the main application, resulting in a functional inconsistency.

To solve this problem, the application uses an **i18n**-type library, specifically designed for web interfaces.

The principle is based on a centralized translation structure in the form of a JavaScript object containing translation keys.

Each interface element that needs to be translated is assigned a **data-i18n** attribute, corresponding to a translation key.

When the application loads, a script automatically detects the browser's language and applies the corresponding translations by dynamically replacing the interface's content.

This approach guarantees linguistic consistency between the 4D application and its web interface.

### Current Limitations

Despite its flexibility, the solution does not yet cover every possible need in terms of advanced sorting.

Three main functional areas still need to be implemented:

- Support for advanced operators,
- Integration of calculation functions or commands into sort criteria.
- Field selection via relations (links).

These features would extend the sort engine to support more complex logic, in particular sorts based on expressions or dynamic calculations.

### Future Feature Enhancements

Several areas for improvement can be considered to enrich and extend the tool's capabilities:

- Adding the missing themes to broaden the possibilities for building criteria.
- Adding new types of criteria to cover more advanced use cases.
- Field selection via relations (links).

- Improving the overall ergonomics of the interface to make the user experience even smoother.
- Extending the composition principle to other functional areas, in particular the building of queries or dynamic filters.

This approach paves the way for a generalization of the concept of **visual composition of business logic**, which could be reused beyond the context of sorting alone, and thus provides a solid foundation for future developments of the application.

## Description of the Hub (Lib\_Mod2)

### localizedString

This method is an internationalization utility. It retrieves a translated string while dynamically inserting parameters into the text. It is particularly useful when a variable value needs to be injected into a translation, for example:

“Delete these 5 records”

In this case, the numeric value cannot be hard-coded in the translation file and must be injected dynamically at execution time.

### isClassicMode

This method is used to determine the application's execution context. It returns an indication of whether the originating window is operating in classic mode or ORDA mode. This test determines the behavior of several internal processes, in particular the application of the sort.

### orderBy

This is the application's main method. It is responsible for opening the sort dialog and initializing the user interface.

Depending on the context passed as a parameter (classic selection or ORDA entity selection), it configures the hub to ensure the sort is handled appropriately.

This method is the main entry point for using the component.

### paletteLoadDictionary

This method is responsible for preparing and providing the web interface with the list of modules available for building sort criteria. It builds a data structure (generally in the form of a JSON object) containing the following elements:

- Available tables
- Associated fields
- Sort directions

Possibly followed by associated metadata.

This information is then sent to the Web Area to dynamically populate the module palette.

### buttonCloseWin

This method is triggered when the user clicks the Close button. Its role is intentionally minimal: it ensures the dialog window is properly closed and the associated resources are released.

### buttonPlansLoad

This method is called when the user selects the Load action. It restores a sort configuration previously saved in .4od format.

The file is loaded from the file system, and its contents are then sent to the web interface in order to visually rebuild the criteria in the editor.

### buttonPlansSave

This method corresponds to the Save action. It serializes all the sort criteria currently defined in the interface and saves them to a file in .4od format.

This mechanism makes it possible to easily keep and reuse complex sort configurations.

### buttonPlansExecute

This method triggers the actual execution of the sort. The criteria built in the interface are interpreted by the class's engine, then applied to the current selection, whether classic or ORDA.

Once the sort has been executed, the data is updated in its original context.

## Conclusion

This sort interface brings a practical and modern enhancement to 4D applications, especially in ORDA-based environments, by offering a more flexible and user-friendly way to manage sorting.

By relying on a **visual, modular, and decoupled** approach, the proposed solution makes it possible to build complex sorts intuitively, without requiring any specific development on the

user's part. Users therefore have a graphical editor capable of faithfully representing the sort logic while retaining a great deal of flexibility in use.

The chosen architecture also guarantees good scalability for the component. The separation between the web interface, the 4D business logic, and the representation of the criteria makes it possible to plan future extensions without calling the system's overall operation into question.